If you want to leverage esbuild plugins, or if you want to use something else, then follow the instructions in the Phoenix Asset Management docs.[22] You can also generate your Phoenix application to not include assets by changing the options passed to `mix phx.new`.[23]

When you are ready to ship to production, it's important to "digest" your assets. Digesting gives a unique hash to each file—based on its contents—so you can easily cache your assets. Your build pipeline should call `mix phx.digest` to achieve this, and Phoenix will include the correct files automatically. Asset instructions are included in the Phoenix deployment docs, so we won't go over it further in this chapter.

Let's switch over to a topic that's frontend-adjacent. We'll look at how to use components to clean up your frontend HTML code.

Use Components to Keep Your User Interface Clean

It's important to give your users consistent style and interaction across your application. Your buttons, modals, lists, tables, and so on should all feel like they belong together. This is done by using consistent CSS styles and JavaScript code across your frontend. You could do this by ensuring perfect HTML and CSS class usage across your application, but that's bound to have inconsistencies appear. Of course, Phoenix supports a better way—components.

Components are self-contained HTML templates, but they offer benefits over normal HTML. Components let you define the attributes that are required to use them, so you can be sure they're used in the proper way across your app. They can do a lot more, but we'll only go over their basics in this chapter.

Components are implemented in modules that include `use Phoenix.Component`. Similar to routers and controllers, the `PhoneAppWeb` module has a function that handles this for us. So, when you see `use PhoneAppWeb, :html`, know that it's pulling in `Phoenix.Component`.

You've already used a component without knowing it. The templates that you copied earlier in the chapter have a `message_form` component that defines the form used to send an SMS message. We'll take this `message_form` component and define the attributes that go into it. Then we'll create a few example components that are separate from our app.

Let's start by defining our attributes!

22. https://hexdocs.pm/phoenix/asset_management.html
23. https://hexdocs.pm/phoenix/Mix.Tasks.Phx.New.html